

Transformer Linear Layer — Training & Inference

Detailed notes from decoder output to next-token prediction

1. Where does the Linear Layer come?

At the end of the Transformer decoder, the decoder produces a contextual representation for every input position. This representation has dimension **d_model**. The Linear Layer is then used as the bridge between this continuous representation and the vocabulary: it converts each d_model-dimensional decoder vector into one score for every word/token in the vocabulary.

Overall flow

Decoder → Decoder Output → Linear Layer → Vocabulary Scores (Logits) → Softmax → Next-token probabilities

2. What does the Linear Layer actually do?

Assume the embedding/model dimension is 8 and the vocabulary contains 10,000 tokens.

Item	Shape	Meaning
Decoder output	$T \times 8$	One 8-dimensional contextual vector per position
Linear weights	$8 \times 10,000$	Maps model dimension to vocabulary dimension
Bias	$1 \times 10,000$	One bias per vocabulary token
Linear output	$T \times 10,000$	One score for every vocabulary token at every position

Mathematically:

$$Y = XW + b$$

For T tokens: $(T \times 8) \times (8 \times 10,000) = T \times 10,000$. The same Linear Layer is applied independently to each position. It does not mix the rows together.

3. Why is the output vocabulary-sized?

The decoder vector contains learned contextual information, but it is not itself a word. The Linear Layer projects that vector into the vocabulary space. If the vocabulary has V tokens, the layer produces V logits. For example, the score for 'I', 'am', 'a', 'student', etc. is produced simultaneously.

Important: Linear output = logits, not probabilities. Softmax is normally applied afterward to turn logits into probabilities.

4. Inference — predicting one word at a time

Suppose the target English sentence is **I am a student**, and we start with the special token **<START>**.

Step	Decoder input	Decoder output	Linear input/output	Prediction
1	<START>	1×8	$1 \times 8 \rightarrow 1 \times 10,000$	I
2	<START> I	2×8	last row $1 \times 8 \rightarrow 1 \times 10,000$	am
3	<START> I am	3×8	last row $1 \times 8 \rightarrow 1 \times 10,000$	a
4	<START> I am a	4×8	last row $1 \times 8 \rightarrow 1 \times 10,000$	student

Key inference idea: Although the decoder produces $T \times 8$, at inference time we only need the representation at the newest/last position to predict the next token. So an implementation can take the last row and send only that 1×8 vector through the Linear Layer.

Example at step 2: decoder produces 2×8 . We select $h_{\blacksquare}$ (the last 1×8 vector), then Linear: $1 \times 8 \rightarrow 1 \times 10,000 \rightarrow \text{Softmax} \rightarrow$ choose 'am'.

5. Why can't we just give the last token to the decoder?

Because the last decoder representation must contain information from the previous tokens. Masked self-attention builds the contextual representation using the available prefix. Therefore, to correctly create $h_{\blacksquare}$ for 'I', the decoder needs the sequence context 'I', not merely the embedding of 'I'.

So distinguish two things: **the decoder needs the context to create the last vector**; after that vector is created, **the Linear Layer only needs the last vector for the next-token prediction during inference**.

6. Training — why do we use all rows?

During training, we normally know the correct target sequence and use teacher forcing. For example, if the target is 'I am a student', the decoder can receive 'I am a' and learn all next-token predictions in parallel:

Decoder position/input	Expected next token
<START>	I
I	am
am	a
a	student

The decoder therefore produces 4 contextual vectors: 4×8 . The Linear Layer maps all of them to $4 \times 10,000$ logits. Each row is compared with the correct target token, and the losses are aggregated. This is computationally efficient because the positions can be processed together.

7. Training vs. inference — the simplest comparison

	Training	Inference
Goal	Learn from the known target sequence	Generate the next token
Decoder output	$T \times d_{\text{model}}$	$T \times d_{\text{model}}$ is naturally produced; only last row is needed

Linear input	All T rows	Usually last row only
Linear output	$T \times \text{vocabulary size}$	$1 \times \text{vocabulary size}$
Prediction	At every position	Only next token
Next decoder input	Uses known target tokens (teacher forcing)	Feeds the model's predicted token back

8. Complete mental picture

Encoder: reads the source sentence and produces encoder representations.

Decoder: uses masked self-attention + encoder-decoder cross-attention + FFN to create contextual representations for the target prefix.

Linear Layer: converts each decoder representation from d_{model} dimensions into vocabulary-sized logits.

Softmax: converts logits into probabilities.

Argmax / sampling: selects the next token.

Autoregressive loop: feeds the selected token back into the decoder and repeats.

9. The most important shape example

Inference, step 2:

Decoder input = 2×8

Decoder output = 2×8

Take last row = 1×8

Linear weights = $8 \times 10,000$

Logits = $1 \times 10,000$

Softmax = $1 \times 10,000$ probabilities

Choose highest-probability token = next word

Training:

Decoder output = 4×8

Linear weights = $8 \times 10,000$

Logits = $4 \times 10,000$

Each row predicts the next target token at that position.

10. Interview-ready answer

“The Linear Layer is the final projection layer after the Transformer decoder. The decoder produces a contextual vector of size d_{model} for each token position. The Linear Layer maps each d_{model} -dimensional vector to vocabulary size, producing one logit per possible token. During training, we usually apply this projection to all decoder positions in parallel because we need a prediction and loss for every target position. During autoregressive inference, we only need the newest decoder representation to predict the next token, so we can select the last row and pass only that vector through the Linear Layer. The logits are then converted to probabilities with softmax, a token is selected, and that token is fed back into the decoder.”